Listing 11.17 Defining a class to test the security policies for `UserController`

```
@WebFluxTest(UserController.class)
@Import(SecurityConfig.class)                    ❶
class UserControllerTests {

  @Autowired
  WebTestClient webClient;

  @MockBean                                      ❷
  ReactiveClientRegistrationRepository clientRegistrationRepository;
}
```

❶ Imports the application's security configuration

❷ A mock bean to skip the interaction with Keycloak when retrieving information about the Client registration

Since we configured Edge Service to return an HTTP `401` response when a request is unauthenticated, let's verify that happens when calling the `/user` endpoint without authenticating first:

```
@Test
void whenNotAuthenticatedThen401() {
  webClient
    .get()
    .uri("/user")
    .exchange()
    .expectStatus().isUnauthorized();
}
```

To test the scenario where a user is authenticated, we can use `mockOidcLogin()`, a configuration object supplied by `SecurityMockServerConfigurers` to mock an OIDC login, synthesize an ID Token, and mutate the request context in `WebTestClient` accordingly.

The `/user` endpoint reads claims from the ID Token through the `OidcUser` object, so we need to build an ID Token with username, first